

04 SQL queries

1. Introduction to SQL Data Manipulation Language (DML)

- SQL DML allows users to **retrieve, insert, delete, and modify data** in relational databases.
 - DML in SQL consists of **four major statements**:
 - **SELECT**: Retrieves data from a database.
 - **INSERT**: Adds new data into tables.
 - **DELETE**: Removes data from tables.
 - **UPDATE**: Modifies existing data in tables.
-

2. SQL SELECT Statement

2.1 Syntax of the SELECT Statement

```
SELECT component  
FROM component  
[WHERE component]  
[GROUP BY component]  
[HAVING component]  
[ORDER BY component];
```

- **SELECT** retrieves data from tables. One or more attributes.
- **FROM** specifies the table to retrieve data from.
- **WHERE** adds conditions to filter data.
- **GROUP BY** groups records into categories.
- **HAVING** filters groups after aggregation.
- **ORDER BY** sorts results.

2.2 Simple Queries

- Example:

```
SELECT SUPNR, SUPNAME FROM SUPPLIER;
```

- The * wildcard select all columns;

```
SELECT * FROM SUPPLIER;
```

Boolean Logic

AND

	true	false	null
true	true	false	null
false	false	false	false
null	null	false	null

OR

	true	false	null
true	true	true	true
false	true	false	null
null	true	null	null

NOT

	NOT
true	false
false	true
null	null

2.3 Queries with Aggregate Functions

- Common aggregate functions:
 - **SUM()**: Total sum of values.
 - **AVG()**: Average value.
 - **COUNT()**: Number of rows.
 - **MIN()** / **MAX()**: Minimum and maximum values.
- Example:

```
SELECT AVG(PURCHASE_PRICE) FROM SUPPLIES;
```

2.4 Queries with GROUP BY and HAVING

- Groups data based on column values.
- **HAVING** filters grouped data.
- Example:

```
SELECT SUPNR, COUNT(PRODNR)
FROM SUPPLIES
GROUP BY SUPNR
HAVING COUNT(PRODNR) > 5;
```

2.5 Queries with ORDER BY

- Sorts results in ascending (**ASC**) or descending (**DESC**) order.
- Example:

```
SELECT SUPNR, SUPNAME
FROM SUPPLIER
ORDER BY SUPNAME ASC;
```

2.6 Join Queries

- Combines rows from two or more tables using common columns.
- **INNER JOIN**:

```
SELECT P.PRODNR, P.PRODNAME, S.SUPNR
FROM PRODUCT P
INNER JOIN SUPPLIES S ON P.PRODNR = S.PRODNR;
```

- **LEFT JOIN** includes all records from the left table:

```
SELECT SUPNR, SUPNAME, PRODNR
FROM SUPPLIER
LEFT JOIN SUPPLIES ON SUPPLIER.SUPNR = SUPPLIES.SUPNR;
```

2.7 Nested Queries (Subqueries)

- Subqueries appear inside another query.
- Example:

```
SELECT SUPNR, SUPNAME
FROM SUPPLIER
WHERE SUPNR IN (SELECT SUPNR FROM SUPPLIES WHERE PRODNR = '0915');
```

2.8 Correlated Queries

- Subqueries that reference outer queries.
- Example:

```
SELECT SUPNR
FROM SUPPLIER S1
WHERE EXISTS (
    SELECT * FROM SUPPLIES S2
    WHERE S1.SUPNR = S2.SUPNR
);
```

2.9 Queries with ALL/ANY

- **ALL** : Compares a value with all rows from a subquery.
- **ANY** : Compares a value with any row from a subquery.
- Example:

```
SELECT SUPNR, SUPNAME
FROM SUPPLIER
WHERE SUPSTATUS > ALL (SELECT SUPSTATUS FROM SUPPLIER WHERE S
UPCITY = 'New York');
```

2.10 Queries with EXISTS

- Checks if a subquery returns any rows.
- Example:

```
SELECT SUPNR, SUPNAME
FROM SUPPLIER
WHERE EXISTS (SELECT * FROM SUPPLIES WHERE SUPPLIER.SUPNR = SUP
PLIES.SUPNR);
```

2.11 Queries with Set Operations

- **UNION**: Combines results from two queries.
- **INTERSECT**: Finds common records between two queries.
- **EXCEPT**: Finds records in the first query but not in the second.
- Example:

```
SELECT SUPNR FROM SUPPLIER
UNION
SELECT SUPNR FROM SUPPLIES;
```

3. SQL INSERT Statement

- Adds new data into tables.
- Example:

```
INSERT INTO PRODUCT (PRODNR, PRODNAME, PRODTYPE, AVAILABLE_QUANTITY)
VALUES ('980', 'Chateau Angelus, 1960', 'red', 6);
```

- Insert multiple rows:

```
INSERT INTO SUPPLIER (SUPNR, SUPNAME, SUPADDRESS, SUPCITY)
VALUES
('132', 'BestWines', '660 Market St', 'New York'),
('201', 'Fine Spirits', '123 Sunset Blvd', 'Los Angeles');
```

4. SQL DELETE Statement

- Removes data from tables.
- Delete all rows:

```
DELETE FROM PRODUCT;
```

- Delete specific rows:

```
DELETE FROM SUPPLIER WHERE SUPSTATUS IS NULL;
```

- Delete with subquery:

```
DELETE FROM SUPPLIES
WHERE PRODNR IN (SELECT PRODNR FROM PRODUCT WHERE PRODNAME
LIKE '%CHARD%');
```

5. SQL UPDATE Statement

- Modifies existing records.
- Example:

```
UPDATE PRODUCT
SET AVAILABLE_QUANTITY = 26
WHERE PRODNR = '0185';
```

- Update multiple rows with CASE:

```
UPDATE SUPPLIER
SET SUPCATEGORY =
CASE
  WHEN SUPSTATUS BETWEEN 70 AND 90 THEN 'GOLD'
  WHEN SUPSTATUS > 90 THEN 'PLATINUM'
  ELSE 'SILVER'
END;
```

6. Example Database Tables

SUPPLIER Table

SUPNR	SUPNAME	SUPADDRESS	SUPCITY	SUPSTATUS
132	BestWines	660 Market Street	New York	50
201	Fine Spirits	123 Sunset Blvd	Los Angeles	75

PRODUCT Table

PRODNR	PRODNAME	PRODTYPE	AVAILABLE_QUANTITY
0154	Chateau Haut Brion 2008	Red	20
0185	Vintage Merlot 2015	Red	26

SUPPLIES Table

SUPNR	PRODNR	PURCHASE_PRICE	DELIV_PERIOD
132	0154	120	30
201	0185	150	45

7. Summary

- SQL DML is **essential for modifying database records**.
 - **SELECT** retrieves data using filtering, grouping, and sorting.
 - **INSERT** adds new records, **DELETE** removes them, and **UPDATE** modifies them.
 - Advanced SQL features like **joins, subqueries, and set operations** enable complex data manipulation.
-

Key Takeaways

- **Mastering SELECT queries is fundamental** for retrieving data efficiently.
 - **Joins and subqueries** are powerful for combining data from multiple tables.
 - **Use WHERE clauses** to filter data, reducing unnecessary computations.
 - **Be cautious with DELETE and UPDATE** to avoid unintended data loss.
 - **Indexing and views** (discussed in later sections) can optimize DML operations.
-

Exercise